

L'ACADÉMIE DE SORCELLERIE

Cahier des charges - FastAPI REST API (Mock Data)

Contexte du projet

Votre équipe est mandatée par le Conseil des Enseignants pour construire l'API qui gèrera l'ensemble de la vie académique de l'Académie : élèves, maisons, cours, examens, compétences magiques, tournois inter-maisons et passage de fin d'année. Le cahier des charges décrit ce que l'API doit permettre de faire, pas comment l'implémenter. L'organisation du code, la structure des fichiers (routeurs, modèles, base mockée) et les choix d'architecture sont entre vos mains.

CONTRAINTES TECHNIQUES

- **Python 3.10+ & FastAPI** pour l'infrastructure d'API REST.
- **Pydantic v2** pour la validation stricte des données et la création des schémas.
- **Base de données en mémoire** : Dictionnaires et listes Python regroupés dans un fichier centralisé `database.py` (aucun ORM / SQLAlchemy pour cette version).
- **Architecture modulaire** : Découpage obligatoire par routeurs (APIRouter) et modèles Pydantic isolés dans leurs dossiers respectifs.
- **Versionning Git** : Un commit par jour minimum avec un message explicite.

FONDATIONS ET COMPTES

Modèle de données en mémoire, CRUD, connexion simulée

OBJECTIF

Poser un modèle de données solide couvrant toute la structure de l'Académie, livrer le CRUD sur les ressources de base, et mettre en place le mécanisme de connexion simulée qui servira toute la semaine.

ENTITÉS ATTENDUES (MODÈLES PYDANTIC & DATA MOCK)

- **Maison** : `id`, `nom`, `couleur`, `fondateur`, `valeurs`.
- **Professeur** : `id`, `nom`, `matiere`, `anciennete`.
- **Cours** : `id`, `intitule`, `niveau_requis`, `capacite_max`, `professeur_id`, `annee_academique`.
- **Élève** : `id`, `nom`, `annee_etudes` (1 à 7), `maison_id`, `familier`, `statut` (actif / diplômé / renvoyé).
- **Utilisateur** : `id`, `email`, `mot_de_passe` (stocké en clair temporairement), `role` (eleve / professeur / admin), `eleve_id` (optionnel), `professeur_id` (optionnel).

CONNEXION SIMULÉE & HEADER X-USER-ID

- **POST /login** : Reçoit email + mot de passe, vérifie la correspondance et renvoie l'utilisateur, son rôle et l'identifiant lié (`eleve_id` ou `professeur_id`).
- Tous les endpoints distinguant "espace élève" / "espace admin" liront un header `X-User-Id`.
- Un appel sans header `X-User-Id` sur un endpoint protégé doit lever une exception `HTTPException(401)` explicite.

EXIGENCES FONCTIONNELLES

- CRUD complet sur Maison, Professeur, Cours, Élève.
- Réponses JSON cohérentes, codes HTTP appropriés (200 OK, 201 Created, 400 Bad Request, 404 Not Found).
- Données initiales (Seed) dans `database.py` : 4 maisons, ~10 élèves, professeurs, cours et comptes de test.

VIE ACADÉMIQUE

Inscriptions, examens, résultats, premier endpoint métier

NOUVELLES ENTITÉS EN MÉMOIRE

- **Inscription** : id, eleve_id, cours_id, date_inscription, statut (inscrit / en_cours / valide / abandonne).
- **Examen** : id, cours_id, titre, date, seuil_reussite.
- **Résultat** : id, eleve_id, examen_id, note.

ENDPOINTS MÉTIER À AJOUTER

- **Inscrire un élève** : POST /cours/{cours_id}/inscrire. Doit refuser (400 Bad Request) si la capacité maximale du cours est atteinte.
- **Résultats en masse** : POST /examens/{examen_id}/resultats avec un payload contenant la liste des notes par élève.
- **Clôturer un examen** : POST /examens/{examen_id}/cloturer. Endpoint métier qui met à jour le statut des inscriptions (valide si note >= seuil).

ESPACE ÉLÈVE / ESPACE ADMIN

- **Élève (via X-User-Id)** : /me/cours, /me/notes (consultation de ses données uniquement).
- **Admin** : Liste filtrable des résultats par cours ou par examen (utilisant des Query Parameters Pydantic).

COMPÉTENCES ET TOURNOIS

Catalogue de compétences, déblocage automatique, duels inter-maisons

COMPÉTENCES & TOURNOIS

- **Compétence** : id, nom, categorie, description, condition de déblocage (examen requis + note min, ou tournoi).
- **Maîtrise** : id, eleve_id, competence_id, date_obtention, source.
- **Tournoi & Duel** : Gestion des affrontements entre deux élèves avec désignation d'un vainqueur.

ENDPOINTS MÉTIER & IDEMPOTENCE

- **Évaluer les compétences** : POST /examens/{examen_id}/evaluer-competences. Attribue les compétences aux élèves éligibles. L'opération doit être **idempotente** (aucun doublon créé si relancé).
- **Clôturer un tournoi** : Détermine le vainqueur global, débloque la compétence associée et augmente la réputation de la maison.
- **Pagination et filtres** : Support des requêtes page et limit sur le listing des compétences et tournois.

PASSAGE DE FIN D'ANNÉE

Endpoint métier majeur, validation Pydantic stricte, fiabilisation

CLÔTURER L'ANNÉE ACADÉMIQUE

Endpoint métier POST /academie/cloturer-annee qui parcourt chaque élève actif:

- Calcule la moyenne générale sur ses inscriptions validées.
- **Promotion** : Si moyenne $\geq$ seuil et année < 7 -> passage à l'année suivante.
- **Redoublement** : Si moyenne $<$ seuil -> l'élève reste dans la même année.
- **Diplomation** : Si moyenne $\geq$ seuil et année $== 7$ -> statut passe à "diplômé".
- **Anti-rejeu** : Refuser de re-clôturer une année déjà clôturée (400 Bad Request).

VALIDATION PYDANTIC & LIVRABLES

- Validation stricte des types et bornes sur tous les formulaires d'écriture (ex. note entre 0 et 20).
- Documentation Swagger UI fonctionnelle et testable sur <http://127.0.0.1:8000/docs>.
- Code structuré avec des routeurs APIRouter isolés par domaine (routers/eleves.py, routers/cours.py, etc.).